

Aufgabe 1

Aufgabe 1.1

Reihenfolge der Token

1. **Javadoc** zuerst
2. **Blockkommentare**
3. **Line-Kommentare**
4. **Strings**
5. **Characters**
6. **Annotationen**
7. **Keywords**
8. **Zahlen**
9. **Identifizier** (optional)

✓ Javadoc

```
/\*\*(  
[  
\\s\\S  
]*?)  
\\*/
```

- `\\*\\*` erkennt `/**`
- `(
[
\\s\\S
]*?)` matcht ALLES, auch Zeilenumbrüche
- non-greedy → stoppt beim ersten `*/`

✓ Blockkommentare

```
/\*([  
^*  
]|\\*+  
[  
^*/  
])  
*\\*/
```

- klassischer Java-Blockkommentar-Regex
- robust gegen `*` im Inhalt

✓ Line-Kommentare

```
//.*
```

- bis zum Zeilenende

✓ Strings

```
"(  
^"  
\\|  
\\. )  
*"
```

- erlaubt escaped quotes `\ "`
- erlaubt escaped backslashes `\\`

✓ Characters

`'([^\\"\\]|\\..)'`

- genau ein Zeichen
- erlaubt escapes wie `'\n'`

✓ Annotationen

`@[A-Za-z][A-Za-z0-9_-]*`

- beginnt mit @
- danach Buchstabe
- dann alphanumerisch, _ oder -

✓ Keywords

`\b(keyword)\b`

- `\b` verhindert Matches wie `className`

Aufgabe 1.2

- ✓ typische Fälle
- ✓ Grenzfälle
- ✓ Regex-Fehlerquellen
- ✓ Escapes in Strings/Chars
- ✓ Kommentare mit Sonderzeichen
- ✓ Keywords nur als ganze Wörter

Für **jede Token-Art** testen :

- Treffer am Anfang / Mitte / Ende
- Mehrere Treffer
- Kein Treffer
- Grenzfälle (z. B. Strings mit `//` oder `/*` drin)

testen **nicht** den Highlighter — nur die **Token-Rege**

Warum 2 und nicht 3

Warum Aufgabe 2 einfacher ist

Der `RegexHighlighter` arbeitet nach einem sehr simplen Prinzip:

1. Alle Token werden unabhängig voneinander auf den Text angewendet.
2. Du bekommst eine große Liste aller Treffer.

3. Du sortierst sie (macht `normalize()` schon für dich).
4. Du entfernst überlappende Regionen (das ist deine Aufgabe).

Das bedeutet:

- keinen Scanner bauen.
- keine Position im Text verwalten.
- keine Token gegeneinander ausspielen.
- keine längsten Matches bestimmen.
- keine Schleifen über den Text schreiben.

Zusammenfassung Aufgabe 2: RegexHighlighter

Ziel der Aufgabe

Der `RegexHighlighter` soll ein naives Syntax-Highlighting umsetzen:

- Alle Tokens werden unabhängig auf den Text angewendet
- Alle Treffer werden gesammelt
- Danach werden Konflikte (Überlappungen) aufgelöst
- Die Template-Methode `computeRegions()` aus `SyntaxHighlighter` steuert den Ablauf

Was wir implementiert haben

✓ 2.1 `collectMatches()`

Diese Methode:

- holt alle Tokens aus `MiniJavaTokens.defaultTokens()`
- wendet jedes Token mit `token.test(text)` auf den Text an
- sammelt alle Treffer in einer Liste
- gibt die Liste zurück (unsortiert, ungefiltert)

✓ 2.2 `resolveConflicts()`

Diese Methode:

- bekommt eine sortierte Liste (durch `normalize()`)
- geht die Liste von vorne nach hinten durch

- behält eine Region nur, wenn sie nicht mit einer bereits übernommenen Region überlappt
- verwirft überlappende Regionen (spätere verlieren gegen frühere)

Wichtig:

- Halb-offene Intervalle `[start, end)`
- `[0, 5)` und `[5, 10)` überlappen nicht
- Bei gleichem Startpunkt gewinnt die längere Region (weil normalize so sortiert)

Wie der RegexHighlighter jetzt arbeitet

Der Ablauf ist:

collectMatches()

1. → alle Treffer aller Tokens

normalize()

→ sortiert nach Start, bei Gleichstand längere zuerst

2. → entfernt ungültige Regionen

resolveConflicts()

→ entfernt überlappende Regionen

3. → Ergebnis ist konfliktfrei und korrekt priorisiert

Tests (Aufgabe 2.3)

Testklasse erstellt, die prüft:

- keine Matches
- einfache Fälle ohne Konflikte
- direkt aufeinanderfolgende Regionen
- Keyword in Kommentar → Kommentar gewinnt
- Javadoc vor Blockkommentar
- String überlappt Keyword → String gewinnt

Aufgabe 4

CI - Pipeline

Die Pipeline soll:

- bei Pull-Requests laufen
- bei workflow_dispatch (manuelle Ausführung) laufen
- NICHT bei Push auf **master** laufen
- drei Jobs enthalten:
 - build → Projekt kompilieren
 - test → JUnit-Tests ausführen
 - format → Spotless im Prüfmodus laufen lassen

Review Saelly05

```
private final Token annotationToken = tokens.get(5);  
private final Token keywordToken = tokens.get(6);
```

Das ist korrekt **für die aktuelle Reihenfolge**, aber:

Wenn jemand später Tokens ergänzt (z. B. Zahlen, Operatoren),

- → müssen die Indizes angepasst werden.

Review-Kommentar:

Die Tests hängen stark von der Reihenfolge der Tokens ab.

Vielleicht wäre es robuster, Tokens über **instanceof** oder Pattern-Vergleich zu identifizieren.

Der PR enthält eine **vollständige Token-Liste** für MiniJava.

Die Reihenfolge der Tokens ist **korrekt priorisiert** (Javadoc → Block → Line → String → Char → Annotation → Keyword).

Die Regex-Ausdrücke sind **syntaktisch korrekt** und decken die geforderten Fälle ab.

Die Tests sind **umfangreich, realistisch** und decken viele Grenzfälle ab.

Die Tests prüfen **Start- und Endpositionen**, was sehr gut ist.

Die Kommentare im Testcode sind **verständlich und hilfreich**.